

UNIVERSIDADE FEDERAL DO PARANÁ
PÓS-GRADUAÇÃO EM INTELIGÊNCIA ARTIFICIAL APLICADA
IAA002 - Linguagem de Programação Aplicada

AMANDA ISABELLY NEUMANN PADILHA

GUSTAVO GOMES MARCONDES

HELEN SARA TEIXEIRA

IGOR NATHAN LOBATO

JEAN ROBERTO REINERT

MATHEUS HENRIQUE MIRANDA

CURITIBA

2026

SUMÁRIO

Imports	3
1. Análise Exploratória dos dados	3
1.a - Importação dos dados	3
1.b - Dados faltantes e tratativa	4
1.c - Verificar dados duplicados	5
1.d - Criar duas categorias, separando colunas numérica e categóricas.	6
1.e - Imprima a contagem de valores por modelo (model) e marca do carro (brand)	9
2. Visualização dos dados	9
2.a - Gere um gráfico da distribuição da quantidade de carros por marca	9
2.b - Gere um gráfico da distribuição da quantidade de carros por tipo de engrenagem do carro	10
2.c - Gere um gráfico da evolução da média de preço dos carros ao longo dos meses de 2022	11
2.d - Gere um gráfico da distribuição da média de preço dos carros por marca e tipo de engrenagem	12
2.f - Gere um gráfico da distribuição da média de preço dos carros por marca e tipo de combustível	13
3. Aplicação de modelos de machine learning para prever o preço médio dos carros	14
3.a - Seleção das variáveis para o modelo	14
3.b - Divisão dos dados em treino e teste	14
3.c - Treinamento dos modelos	15
3.d - Geração das previsões	15
3.e - Análise de importância das variáveis	16
3.g - Avaliação dos modelos	17

Imports

```
# Biblioteca Pandas - Manipulação de dados
import pandas as pd

# Biblioteca Seaborn - Criação de gráficos
import seaborn as sns

# Biblioteca Matplotlib - Criação de gráficos
import matplotlib.pyplot as plt

# Biblioteca para ignorar mensagens de warning (aviso) ao rodar uma
# célula de código
import warnings
warnings.filterwarnings('ignore')

# Importações necessárias
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Importando modelos
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
```

1. Análise Exploratória dos dados

1.a - Importação dos dados

```
# Função read_csv para importar os dados da pasta do computador
dados = pd.read_csv('precos_carros_brasil.csv')

# Listando o nome das colunas
dados.columns
```

Resultado:

```
...   Index(['year_of_reference', 'month_of_reference', 'fiipe_code',
            'authentication', 'brand', 'model', 'fuel', 'gear', 'engine_size',
            'year_model', 'avg_price_brl'],
            dtype='str')
```

Generate

+ Code

+ Markdown

```
dados.head()
```

Resultado:

...	year_of_reference	month_of_reference	fipe_code	authentication	brand	model	fuel	gear	engine_size	year_model	avg_price_brl
0	2021.0	January	004001-0	cfzltzfwrcp	GM - Chevrolet	Corsa Wind 1.0 MPFI / EFI 2p	Gasoline	manual	1	2002.0	9162.0
1	2021.0	January	004001-0	cdqwxwpw3y2p	GM - Chevrolet	Corsa Wind 1.0 MPFI / EFI 2p	Gasoline	manual	1	2001.0	8832.0
2	2021.0	January	004001-0	cb1t3xwwj1xp	GM - Chevrolet	Corsa Wind 1.0 MPFI / EFI 2p	Gasoline	manual	1	2000.0	8388.0
3	2021.0	January	004001-0	cb9gct6j65r0	GM - Chevrolet	Corsa Wind 1.0 MPFI / EFI 2p	Alcohol	manual	1	2000.0	8453.0
4	2021.0	January	004003-7	g15wg0gbz1fx	GM - Chevrolet	Corsa Pick-Up GL/ Champ 1.6 MPFI / EFI	Gasoline	manual	1,6	2001.0	12525.0

```
dados.shape
```

Resultado:

```
(267542, 11)
```

1.b - Dados faltantes e tratativa

```
# Verificando valores faltantes nos dados
dados.isna().any()
```

Resultado:

```
year_of_reference    True
month_of_reference   True
fipe_code            True
authentication       True
brand               True
model               True
fuel                True
gear                True
engine_size         True
year_model          True
avg_price_brl       True
dtype: bool
```

```
# Todas as colunas possuem valores faltantes, vamos verificar a
quantidade de valores faltantes em cada coluna
dados.isna().sum()
```

Resultado:

```
year_of_reference    65245
month_of_reference   65245
fipe_code            65245
authentication       65245
brand               65245
model               65245
fuel                65245
gear                65245
engine_size         65245
year_model          65245
avg_price_brl       65245
dtype: int64
```

```
# Removendo os valores faltantes da coluna principal 'model'
dados.dropna(subset=['model'], inplace=True)
# Verificando novamente os valores faltantes nos dados
dados.isna().sum()
```

Resultado:

```
year_of_reference      0
month_of_reference     0
fipecode               0
authentication         0
brand                  0
model                  0
fuel                   0
gear                   0
engine_size            0
year_model             0
avg_price_brl          0
dtype: int64
```

```
# Conferindo o número de linhas e colunas do dataframe após a limpeza
dados.shape
```

Resultado:

```
(202297, 11)
```

1.c - Verificar dados duplicados

```
# Soma a quantidade de dados duplicados (linhas que possuem os mesmos
valores em todas as colunas)
dados.duplicated().sum()
```

Resultado:

```
np.int64(2)
```

```
# Exibindo os dados duplicados
dados[dados.duplicated()].head()
```

Resultado:

year_of_reference	month_of_reference	fipecode	authentication	brand	model	fuel	gear	engine_size	year_model	avg_price_brl	
45793	2021.0	June	025232-8	5rtdwkpqpq5h	Renault	DUSTER OROCH Dyna. 2.0 Flex 16V Mec.	Gasoline	manual	2	2018.0	69893.0
189896	2022.0	December	003296-4	3r6c277cnqcb	Ford	Ranger Limited 3.0 PSE 4x4 CD TB Diesel	Diesel	manual	3	2007.0	64638.0

```
# Removendo os dados duplicados do dataframe
dados.drop_duplicates(inplace=True)
# Verificando novamente a quantidade de dados duplicados sem a coluna
'authentication' que não é categorica para os carros
dados.duplicated(subset=dados.columns.difference(['authentication'])).sum()
```

Resultado:

```
np.int64(0)
```

```
# Conferindo o número de linhas e colunas do dataframe após a limpeza
dados.shape
```

Resultado:

```
(202295, 11)
```

1.d - Criar duas categorias, separando colunas numérica e categóricas.

```
# Verificando o tipo de dados de cada coluna
dados.dtypes
```

Resultado:

```
year_of_reference    float64
month_of_reference    str
fipecode              str
authentication        str
brand                 str
model                 str
fuel                  str
gear                  str
engine_size           str
year_model            float64
avg_price_brl         float64
dtype: object
```

```
# O engine_size está como string, mas aparentemente é numérico.
dados['engine_size'].unique()
```

Resultado:

```
<StringArray>
[ '1', '1,6', '2,2', '4,3', '2,5', '1,8',  '2', '4,2', '3,8', '4,1', '5,7',
  '2,8', '2,4', '1,4', '3,6', '6,2',  '3', '1,2', '1,5', '1,3', '1,9', '2,3',
   '4', '3,9',  '5', '3,5', '3,2', '2,7', '3,3']
Length: 29, dtype: str
```

```
# Convertendo o separador decimal de vírgula para ponto e depois
converter a coluna para numérica
```

```
dados['engine_size'] = dados['engine_size'].str.replace(',',
'.').astype('float64')
```

```
# Ano está como float mas o correto seria inteiro
```

```
dados['year_of_reference'].unique()
```

Resultado: array([2021., 2022., 2023.])

```
# Convertendo a coluna 'year_of_reference' e 'year_model' para o tipo inteiro
dados['year_of_reference'] = dados['year_of_reference'].astype('int64')
dados['year_model'] = dados['year_model'].astype('int64')
# month_of_reference também pode ser convertida em numérica
dados['month_of_reference'].unique()
```

Resultado:

```
<StringArray>
[ 'January', 'February', 'March', 'April', 'May', 'June',
  'July', 'August', 'September', 'October', 'November', 'December']
Length: 12, dtype: str
```

Função

```
def convert_month_to_numeric(month):
    if month == 'January':
        return 1
    elif month == 'February':
        return 2
    elif month == 'March':
        return 3
    elif month == 'April':
        return 4
    elif month == 'May':
        return 5
    elif month == 'June':
        return 6
    elif month == 'July':
        return 7
    elif month == 'August':
        return 8
    elif month == 'September':
        return 9
    elif month == 'October':
        return 10
    elif month == 'November':
        return 11
    elif month == 'December':
        return 12
    else:
        return None

dados['month_of_reference_num'] = dados['month_of_reference'].apply(convert_month_to_numeric)
```

```
# conferindo a tipagem dos dados após a conversão
dados.dtypes
```

Resultado:

```
year_of_reference      int64
month_of_reference      str
fipecode                str
authentication          str
brand                   str
model                   str
fuel                    str
gear                    str
engine_size             float64
year_model              int64
avg_price_brl           float64
month_of_reference_num  int64
dtype: object
```

```
# Criando categorias para separar numéricas e categóricas
numericas_cols = [col for col in dados.columns if dados[col].dtype !=
'str']
categoricas_cols = [col for col in dados.columns if dados[col].dtype ==
'str']
```

```
dados[numericas_cols].describe()
```

Resultado:

	year_of_reference	engine_size	year_model	avg_price_brl	month_of_reference_num
count	202295.000000	202295.000000	202295.000000	202295.000000	202295.000000
mean	2021.564695	1.822302	2011.271514	52756.765713	6.295811
std	0.571904	0.734432	6.376241	51628.912116	3.552728
min	2021.000000	1.000000	2000.000000	6647.000000	1.000000
25%	2021.000000	1.400000	2006.000000	22855.000000	3.000000
50%	2022.000000	1.600000	2012.000000	38027.000000	6.000000
75%	2022.000000	2.000000	2016.000000	64064.000000	9.000000
max	2023.000000	6.200000	2023.000000	979358.000000	12.000000

```
dados[categoricas_cols].describe()
```

Resultado:

	month_of_reference	fipecode	authentication	brand	model	fuel	gear
count	202295	202295	202295	202295	202295	202295	202295
unique	12	2091	202295	6	2112	3	2
top	January	001216-5	cfzltzfwrcp	Fiat	Palio Week. Adv/Adv TRYON 1.8 mpi Flex	Gasoline	manual
freq	24260	425	1	44962	425	168684	161883

1.e - Imprima a contagem de valores por modelo (model) e marca do carro (brand)

```
# Exibindo a contagem de valores por modelo:
modelos_contagem = dados['model'].value_counts()
print(modelos_contagem)
```

Resultado:

```
model
Palio Week. Adv/Adv TRYON 1.8 mpi Flex    425
Focus 1.6 S/SE/SE Plus Flex 8V/16V 5p    425
Focus 2.0 16V/SE/SE Plus Flex 5p Aut.    400
Saveiro 1.6 Mi/ 1.6 Mi Total Flex 8V     400
Corvette 5.7/ 6.0, 6.2 Targa/Stingray    375
...
STEPWAY Zen Flex 1.0 12V Mec.             2
Saveiro Robust 1.6 Total Flex 16V CD      2
Saveiro Robust 1.6 Total Flex 16V         2
Gol Last Edition 1.0 Flex 12V 5p          2
Polo Track 1.0 Flex 12V 5p                2
Name: count, Length: 2112, dtype: int64
```

```
# Exibindo a contagem de valores por marca:
marcas_contagem = dados['brand'].value_counts()
print(marcas_contagem)
```

Resultado:

```
brand
Fiat          44962
VW - Volkswagen 44312
GM - Chevrolet 38590
Ford          33150
Renault       29191
Nissan        12090
Name: count, dtype: int64
```

2. Visualização dos dados

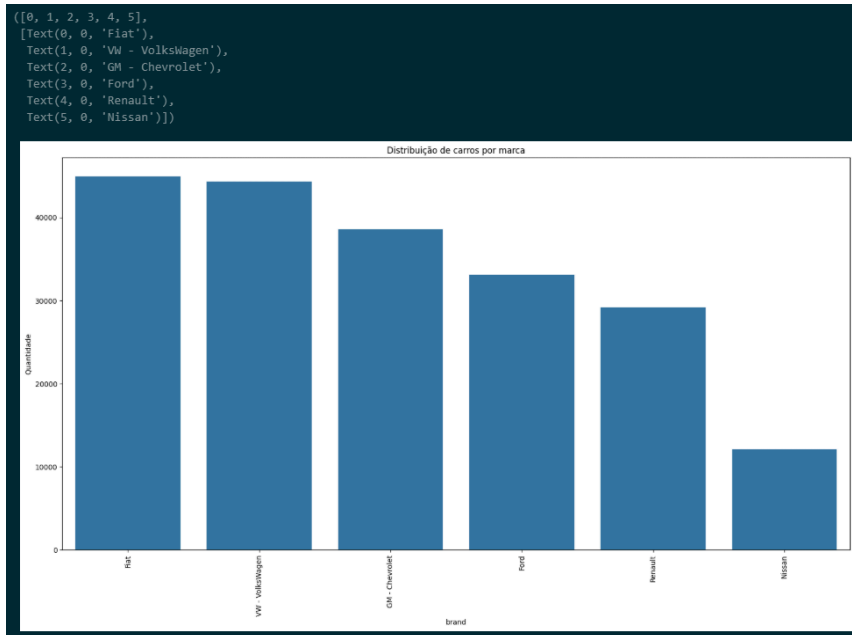
2.a - Gere um gráfico da distribuição da quantidade de carros por marca

```
# Exibindo a contagem de valores por tipo de cambio:
valores_marca = dados['brand'].value_counts()

plt.figure(figsize=(20,10))
sns.barplot(x=valores_marca.index, y=valores_marca.values)

plt.title('Distribuição de carros por marca')
plt.ylabel('Quantidade')
plt.xticks(rotation=90)
```

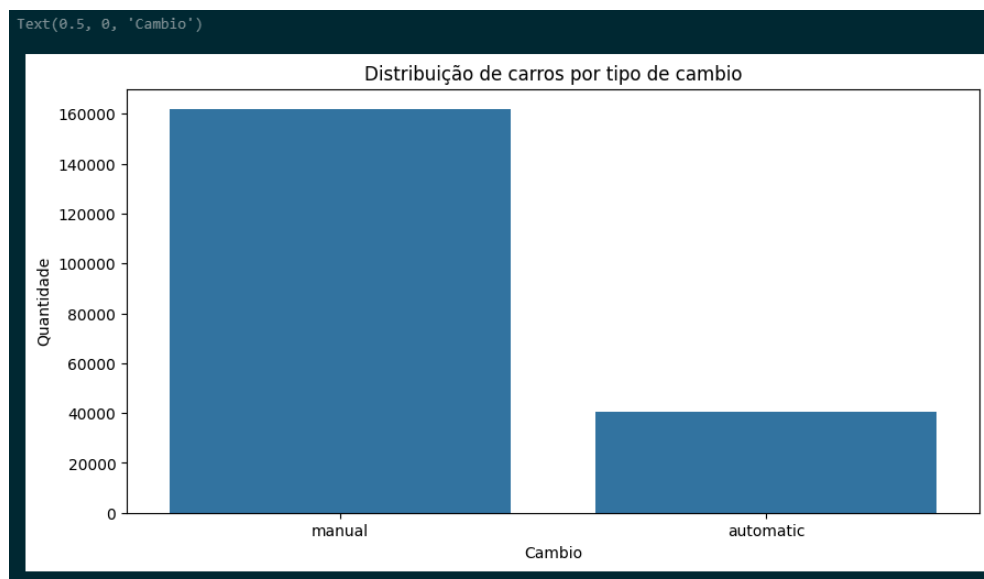
Resultado:



2.b - Gere um gráfico da distribuição da quantidade de carros por tipo de engrenagem do carro

```
# Exibindo a contagem de valores por tipo de cambio:  
valoresCambio = dados['gear'].value_counts()  
  
plt.figure(figsize=(10,5))  
sns.barplot(x=valoresCambio.index, y=valoresCambio.values)  
plt.title('Distribuição de carros por tipo de cambio')  
plt.ylabel('Quantidade')  
plt.xlabel('Cambio')
```

Resultado:



2.c - Gere um gráfico da evolução da média de preço dos carros ao longo dos meses de 2022

```
# Analisando a distribuição dos dados de referência por ano
dados_2022 = dados[dados['year_of_reference'] == 2022]
dados_2022[['year_of_reference']].describe()
```

Resultado:

year_of_reference	
count	97797.0
mean	2022.0
std	0.0
min	2022.0
25%	2022.0
50%	2022.0
75%	2022.0
max	2022.0

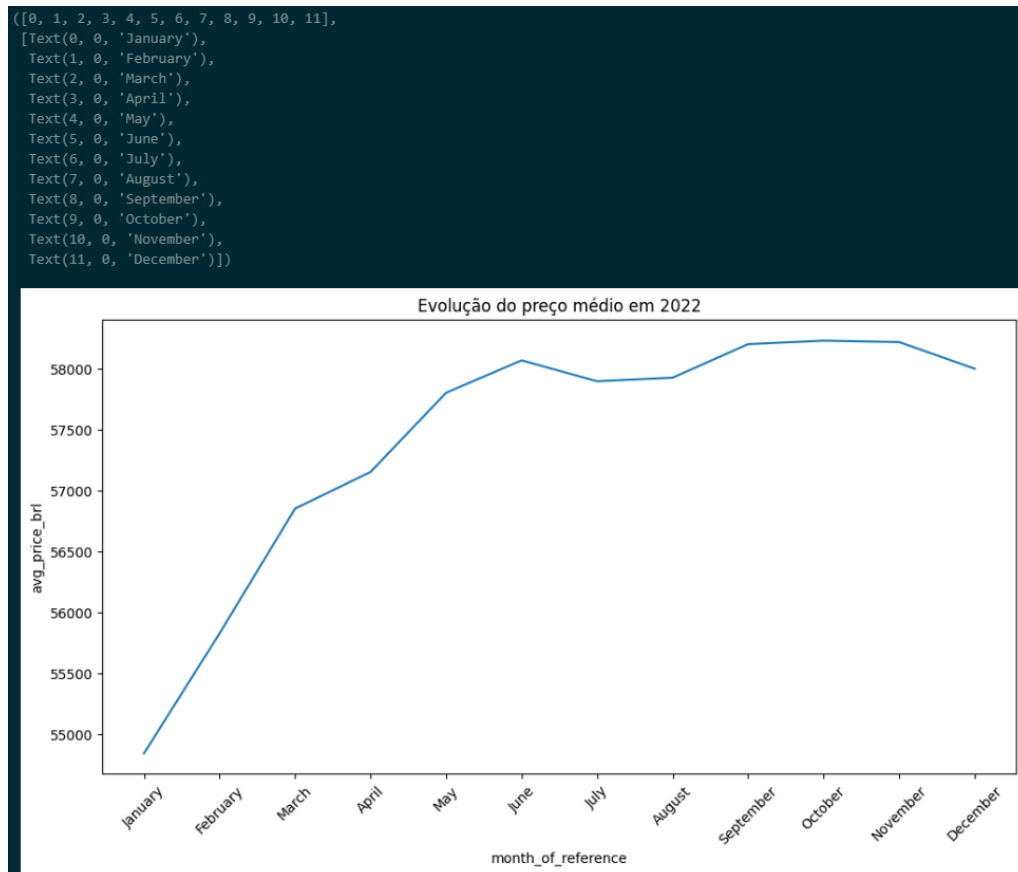
```
# Analisando a evolução do preço médio dos carros ao longo dos meses de 2022
media_preco_mes =
dados_2022.groupby('month_of_reference')['avg_price_brl'].mean().reset_index()

# Adicione esta parte para ordenar por mês numérico
media_preco_mes['month_of_reference_num'] =
media_preco_mes['month_of_reference'].apply(convert_month_to_numeric)
media_preco_mes =
media_preco_mes.sort_values(by='month_of_reference_num')

plt.figure(figsize=(12,6))
sns.lineplot(x='month_of_reference', y='avg_price_brl',
data=media_preco_mes)

plt.title('Evolução do preço médio em 2022')
plt.xticks(rotation=45)
```

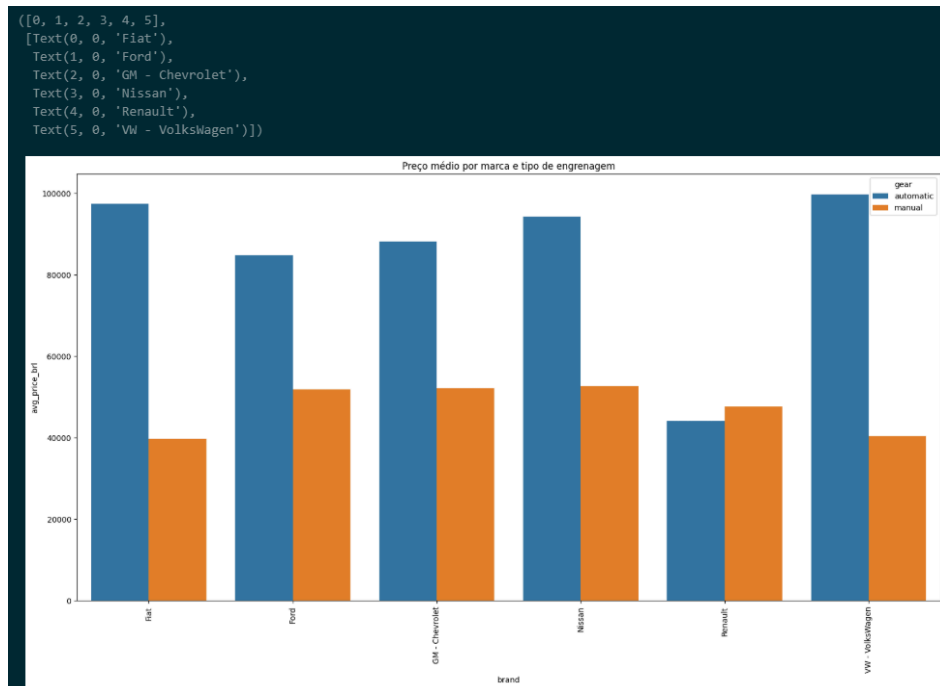
Resultado:



2.d - Gere um gráfico da distribuição da média de preço dos carros por marca e tipo de engrenagem

```
# Analisando o preço médio por marca e tipo de engrenagem  
media_marca_gear = dados.groupby(['brand',  
 'gear'])['avg_price_brl'].mean().reset_index()  
  
plt.figure(figsize=(20,10))  
sns.barplot(x='brand', y='avg_price_brl', hue='gear',  
 data=media_marca_gear)  
  
plt.title('Preço médio por marca e tipo de engrenagem')  
plt.xticks(rotation=90);
```

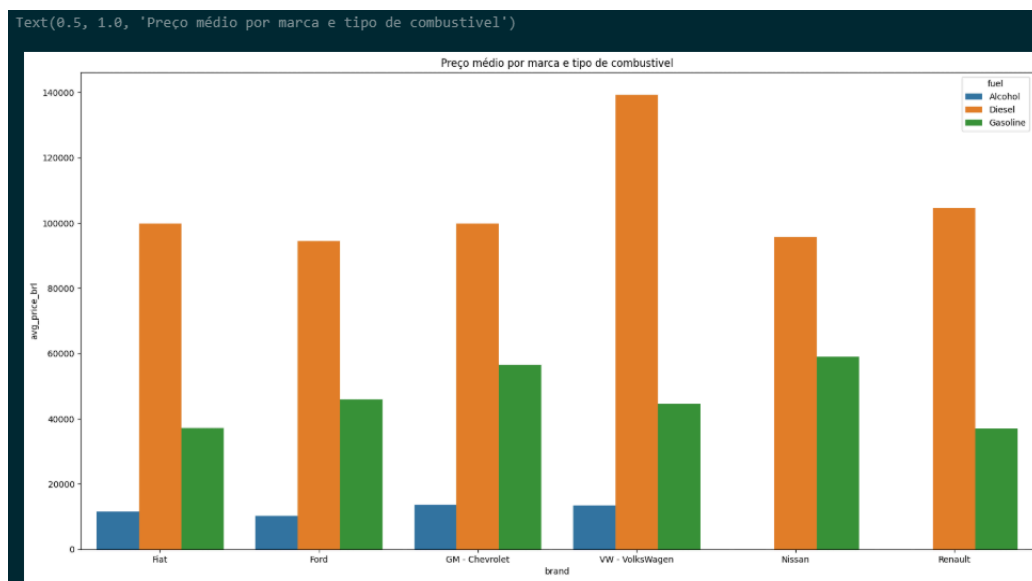
Resultado:



2.f - Gere um gráfico da distribuição da média de preço dos carros por marca e tipo de combustível

```
precoCarroComb = dados.groupby(['fuel',  
 'brand'])['avg_price_brl'].mean().reset_index()  
  
plt.figure(figsize=(20,10))  
sns.barplot(x='brand', y='avg_price_brl', hue='fuel',  
 data=precoCarroComb)  
plt.title('Preço médio por marca e tipo de combustivel')
```

Resultado:



3. Aplicação de modelos de machine learning para prever o preço médio dos carros

3.a - Seleção das variáveis para o modelo

```
# Variável target
y = dados['avg_price_brl']

# Variáveis independentes
X =
dados[['year_of_reference', 'engine_size', 'year_model', 'month_of_referen
ce_num', 'brand', 'fuel', 'gear']]
```

```
# Separação entre colunas numéricas e categóricas

numericas =
['year_of_reference', 'engine_size', 'year_model', 'month_of_reference_num
']
categóricas = ['brand', 'fuel', 'gear']
```

```
# One Hot Encoding para variáveis categóricas

preprocessador = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categóricas)
    ],
    remainder='passthrough'
)
```

3.b - Divisão dos dados em treino e teste

```
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42
)
```

3.c - Treinamento dos modelos

```
# Pipeline Random Forest
```

```

pipeline_rf = Pipeline(steps=[
    ('prep', preprocessador),
    ('modelo', RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ))
])

```

```

# Pipeline XGBoost

pipeline_xgb = Pipeline(steps=[
    ('prep', preprocessador),
    ('modelo', XGBRegressor(
        n_estimators=100,
        learning_rate=0.1,
        random_state=42
    ))
])

```

3.d - Geração das previsões

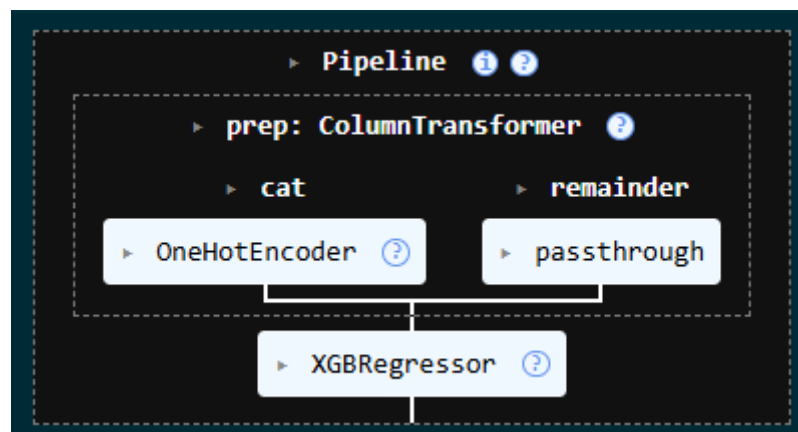
```

# Treinando os modelos

pipeline_rf.fit(X_treino, y_treino)
pipeline_xgb.fit(X_treino, y_treino)

```

Resultado:



```

# Predições

pred_rf = pipeline_rf.predict(X_teste)

```

```
pred_xgb = pipeline_xgb.predict(X_teste)
```

3.e - Análise de importância das variáveis

```
encoder = pipeline_rf.named_steps['prep'].named_transformers_['cat']
nomes_categoricos = encoder.get_feature_names_out(categoricas)

nomes_variaveis = list(nomes_categoricos) + numericas
```

```
# Importância Random Forest

importancias_rf =
pipeline_rf.named_steps['modelo'].feature_importances_

importancia_rf_df = pd.DataFrame({
    'variavel': nomes_variaveis,
    'importancia': importancias_rf
}).sort_values(by='importancia', ascending=False)

importancia_rf_df.head(10)
```

Resultado:

	variavel	importancia
12	engine_size	0.423056
13	year_model	0.391407
7	fuel_Diesel	0.074723
10	gear_manual	0.020154
9	gear_automatic	0.019550
11	year_of_reference	0.013723
8	fuel_Gasoline	0.013040
14	month_of_reference_num	0.012368
5	brand_VW - VolksWagen	0.011335
3	brand_Nissan	0.010370

```
# Importância XGBoost
```



```
importancias_xgb =
pipeline_xgb.named_steps['modelo'].feature_importances_

importancia_xgb_df = pd.DataFrame({
    'variavel': nomes_variaveis,
    'importancia': importancias_xgb
}).sort_values(by='importancia', ascending=False)

importancia_xgb_df.head(10)
```

Resultado:

	variavel	importancia
7	fuel_Diesel	0.249878
12	engine_size	0.243261
13	year_model	0.203904
3	brand_Nissan	0.128753
9	gear_automatic	0.052916
4	brand_Renault	0.037764
5	brand_VW - VolksWagen	0.031270
11	year_of_reference	0.016465
0	brand_Fiat	0.015986
2	brand_GM - Chevrolet	0.007738

3.g - Avaliação dos modelos

```
# Random Forest

mse_rf = mean_squared_error(y_teste, pred_rf)
mae_rf = mean_absolute_error(y_teste, pred_rf)
r2_rf = r2_score(y_teste, pred_rf)

print("Random Forest")
print("MSE:", mse_rf)
print("MAE:", mae_rf)
print("R2:", r2_rf)
```

Resultado:

```
Random Forest  
MSE: 141545606.7453853  
MAE: 5858.226771635237  
R2: 0.9474053983875719
```

```
# XGBoost  
  
mse_xgb = mean_squared_error(y_teste, pred_xgb)  
mae_xgb = mean_absolute_error(y_teste, pred_xgb)  
r2_xgb = r2_score(y_teste, pred_xgb)  
  
print("XGBoost")  
print("MSE:", mse_xgb)  
print("MAE:", mae_xgb)  
print("R2:", r2_xgb)
```

Resultado:

```
XGBoost  
MSE: 109882261.70527738  
MAE: 5680.408439638342  
R2: 0.9591706594676769
```